

第1回 AIプログラミング1

環境構築とPythonの基礎

大西朔永

AIプログラミング1の概要

目的

- AI技術の理解にはPythonの読み書きが重要



- Pythonの基礎的な文法からデータ構造・クラスまで学習
 - 変数、条件文、繰り返し
 - リスト、辞書
 - 関数、クラス

データの読み込み・変換処理のプログラムが作成できるようになることが目標

達成目標

- Pythonの文法を理解できる
- プログラムを他の人に説明できる
- Pythonを用いたプログラム開発の基礎を習得する

成績評価

毎回の課題をあわせて40点で評価



期末試験は60点で評価



総計で60点以上を合格とする

講義の構成

回	内容
1	環境構築とPythonの基礎
2	変数と代入
3	条件文
4	繰り返し
5	リストの基礎
6	リストの応用処理
7	辞書の基礎
8	辞書の応用処理

回	内容
9	ファイル処理
10	関数の基礎
11	関数の応用
12	関数とデータ構造の組み合わせ
13	クラスの基礎
14	ライブラリとクラスの活用
15	総合演習

第1回の位置と内容

回	内容
1	環境構築とPythonの基礎
2	変数と代入
3	条件文
4	繰り返し
5	リストの基礎
6	リストの応用処理
7	辞書の基礎
8	辞書の応用処理

回	内容
9	ファイル処理
10	関数の基礎
11	関数の応用
12	関数とデータ構造の組み合わせ
13	クラスの基礎
14	ライブラリとクラスの活用
15	総合演習

開発環境の紹介（3種類）

Google Colaboratory

- メイン環境
- オンライン環境
- ブラウザ上で動作
- Googleアカウントで利用可能

VSCode

- ローカル環境
- ローカルPC上で動作
- .pyファイルで開発

Cloud Shell Editor

- オンライン環境
- ブラウザ上のVSCode
- .pyファイルで開発

プログラミングの学び方

推奨の学び方

1. 動画を視聴
2. 演習部分（プログラム作成）を実施
3. テストを実施
4. 復習テストを実施
（実際にそのプログラムを作成することを推奨）

プログラムは**自分の手で入力して実行する**ことが重要

Pythonとは

Pythonとは？

- 読みやすく書きやすいプログラミング言語

特徴

- シンプルな文法
- 豊富なライブラリ
- 幅広い用途

なぜ学ぶのか

- AI・データサイエンスの標準言語
- 初学者にも学びやすい

基礎プログラミングで学んだC言語の知識を活かして
Pythonを学ぶ

インタプリタとコンパイラ

C言語（コンパイラ）

1. ソースコードを記述
2. コンパイル（翻訳）
3. 実行ファイルを生成
4. 実行

実行速度が速い

Python（インタプリタ）

1. ソースコードを記述
2. 1行ずつ翻訳しながら
実行

開発効率が高い

C言語: コンパイル → 実行（2段階） ⇔ Python: そのまま実行（1段階）

Pythonの思想とインデント

The Zen of Python

- 美しいことはよいことだ
- 読みやすさが重要
- シンプルは複雑に勝る

インデントによるブロック構造

C言語: { } で囲む

```
if (x > 0) {  
    ...printf("positive");  
}
```

Python: インデントで表現

```
if x > 0:  
    ...print("positive")
```

Pythonではインデント（字下げ）がプログラムの構造を決定

Pythonの用途

データサイエンス

データ分析・可視化

NumPy, pandas,
Matplotlib

AI・機械学習
深層学習・自然
言語処理

TensorFlow,
PyTorch,
scikit-learn

Web開発

Webアプリケーション

Django, Flask,
FastAPI

AI分野ではPythonが事実上の標準言語

外部パッケージとエコシステム

- Pythonの強みは豊富な外部パッケージ（ライブラリ）
- パッケージ管理ツールでインストール・管理

pip

- Python標準のパッケージ管理
 - `pip install パッケージ名`

uv

- 高速なパッケージ管理
 - `uv add パッケージ名`

授業での使い分け:

Colabでは**pip**、VSCode/Cloud Shell Editorでは**uv**

仮想環境

- プロジェクトごとに独立したPython環境を作成する仕組み
- パッケージのバージョン競合を防止

仮想環境なし

Project A: パッケージX v1.0
Project B: パッケージX v2.0
→ 競合が発生

仮想環境あり

Project A: 専用環境 v1.0
Project B: 専用環境 v2.0
→ 独立して管理

授業ではuvが仮想環境を自動管理

Pythonの実行方式

インタラクティブ実行

- 1行ずつ入力して即座に結果確認
- 動作確認や学習に便利

```
>>> 1 + 2
3
>>> print("Hello")
Hello
```

.pyファイル実行

- ファイルに保存して実行
- 本格的な開発に使用

```
$ python script.py
Hello, World!
```

Jupyter Notebook

- セル単位でコード実行
- 結果を即座に確認
- **Google Colabはこの形式**

Google Colaboratory

Google Colaboratoryとは

- Googleが提供する無料のクラウド実行環境
- Jupyter Notebookをベースとしたサービス
- ブラウザ上で動作（インストール不要）

Googleアカウントがあれば
すぐに利用可能

Google Driveと連携
ノートブックの保存・
共有が容易

本講義のメイン開発環境

Colabの始め方（デモ）

1. Googleアカウントでログイン
2. Google Colabにアクセス
3. 「ノートブックを新規作成」をクリック

デモで実際に操作を確認

ノートブックの基本操作（デモ）

コードセル

- Pythonコードを記述
- `Ctrl + Enter` で実行
- 実行結果がその下に表示

テキストセル

- Markdownで記述
- メモや説明を記載
- プログラムの解説に使用

基本操作

- セルの作成: 「+ コード」「+ テキスト」ボタン
- セルの実行: `Ctrl + Enter`
- セルの削除: セル右上のメニュー → ゴミ箱マーク

はじめてのPythonプログラム（デモ）

ai_programming_1_01_01.py

```
1  # print文で文字列を表示するプログラム
2
3  # 文字列の表示
4  print("Hello, World!")
5
6  # 複数行の表示
7  print("Pythonへようこそ！")
8  print("AIプログラミング1を始めましょう。")
9
10 # コメントは実行されない
11 # print("この行は表示されない")
```

実行結果

```
Hello, World!↵
Pythonへようこそ！↵
AIプログラミング1を始めましょう。↵
```

- `print()`: 文字列を表示する関数
- `#` から始まる行はコメント（実行されない）

.ipynbと.pyの違い

.ipynb（ノートブック形式）

- コードセル＋テキストセル
- 実行結果をファイル内に保持
- Colabのメイン形式

.py（スクリプト形式）

- コードのみを記述
- ターミナルから実行
- VSCode / Cloud Shell Editorのメイン形式

どちらの環境でも.ipynbと.pyの両方を扱える
Colabからは「ファイル」→「.pyをダウンロード」で変換可能

.pyファイルの実行（デモ）

Colabで.pyファイルを実行する手順

1. .pyファイルをColabにアップロード
2. コードセルで以下を実行

```
!python script.py
```

- `!` を先頭に付けるとLinuxコマンドを実行可能
- `.py` を作成・編集することも可能

四則演算と結果の表示

ai_programming_1_01_02. py

```
1  # 四則演算と結果の表示
2
3  # 足し算
4  print(10 + 3)
5
6  # 引き算
7  print(10 - 3)
8
9  # 掛け算
10 print(10 * 3)
11
12 # 割り算 (小数)
13 print(10 / 3)
14
15 # 割り算 (整数)
```

実行結果

```
13↵
7↵
30↵
3.3333333333333335↵
3↵
1↵
1024↵
```

- / : 割り算 (結果は小数)
- // : 割り算 (結果は整数)
- % : 余り
- ** : べき乗

四則演算と結果の表示

ai_programming_1_01_02.py

```
9  #・掛け算
10 print(10*3)
11
12 #・割り算（小数）
13 print(10/3)
14
15 #・割り算（整数）
16 print(10//3)
17
18 #・余り
19 print(10%3)
20
21 #・べき乗
22 print(2**10)
```

実行結果

```
13↵
7↵
30↵
3.3333333333333335↵
3↵
1↵
1024↵
```

- / : 割り算（結果は小数）
- // : 割り算（結果は整数）
- % : 余り
- ** : べき乗

VSCode

VSCodeとは

- Microsoftが開発した無料のコードエディタ
- 拡張機能で多くの言語に対応
- ローカルPC上で動作

**ローカル環境での
本格的な開発に最適**

インストール方法は
公式サイトからダウン
ロード

uvのインストールとPython導入（デモ）

1. uvのインストール

- 公式サイトでのインストールコマンドを実行（デモ）
- <https://docs.astral.sh/uv/getting-started/installation/>

2. Pythonの導入

Pythonは `uv sync` 時に自動インストールされる
（`pyproject.toml` の設定に基づく）

プロジェクト環境の構築（デモ）

配布ファイル

- `pyproject.toml`: プロジェクト設定・依存関係の定義
- `.vscode/`: VSCode設定（推奨拡張機能等）

セットアップ手順

1. 配布フォルダをVSCodeで開く
2. `uv sync` で依存関係（Ruff等）を同期
3. 推奨拡張機能を一括インストール

パッケージの追加（デモ）

- 外部パッケージが必要な場合は `uv add` で追加

```
uv · add · パッケージ名
```

例: NumPyの追加

```
uv · add · numpy
```

`pyproject.toml` に自動的に依存関係が追記される

Colabでは `pip install` パッケージ名 を使用

.pyファイルの作成と実行（デモ）

手順

1. VSCodeで新規ファイルを作成（拡張子 `.py`）
2. Pythonコードを記述
3. ターミナルから実行

```
uv · run · python · ファイル名.py
```

VSCodeの右上「▶ Run Python File」ボタンでも実行可能

文字列の連結と表示

ai_programming_1_01_03.py

```
1  # 文字列の連結と表示
2
3  # 文字列の連結
4  greeting = "こんにちは" + "," + "Python!"
5  print(greeting)
6
7  # 文字列の繰り返し
8  line = "=" * 20
9  print(line)
10
11 # 数値と文字列の組み合わせ
12 name = "Python"
13 version = 3
14 print(name + "のバージョンは" + str(version) + "です。")
```

実行結果

```
こんにちは、Python !
=====
Pythonのバージョンは3です。
```

- + : 文字列の連結
- * : 文字列の繰り返し
- str() : 数値を文字列に変換

Colabとの違い

Google Colab

- ブラウザ上で動作
- ノートブック形式 (.ipynb)
- セル単位で実行
- Googleアカウント必須
- パッケージ管理: pip

VSCode

- ローカルPCで動作
- スクリプト形式 (.py)
- ファイル全体を実行
- インストールが必要
- パッケージ管理: uv

どちらでも同じPythonプログラムを実行可能

Google Cloud Shell Editor

Cloud Shell Editorとは

- Googleが提供するブラウザ上のVSCode環境
- Eclipse Theiaベース（VSCodeと同様のUI）
- Googleアカウントがあれば利用可能

**インストール不要で
VSCodeと同様の操作感**

**.pyファイルで開発
ターミナルからコマン
ド実行**

VSCodeのエディタ部分はオープンソース（Code - OSS）として公開
Eclipse Theiaはこれと同じ技術基盤を利用したエディタフレームワーク

Cloud Shell Editorの始め方（デモ）

1. Google Cloud Shellにアクセス（デモ）
 - <https://shell.cloud.google.com/?hl=ja&show=ide>
2. VSCodeに似た画面が開く

デモで実際に操作を確認

プロジェクトのセットアップ（デモ）

手順

1. 配布ファイル（`pyproject.toml`, `.vscode/`）をフォルダに配置
2. Cloud Shell Editorでそのフォルダを開く
3. ターミナルで `uv sync` を実行
4. 推奨拡張機能をインストール

Cloud Shell Editorにはuv・Pythonがデフォルトでインストール済み

.py実行（デモ）

ターミナルからの実行

```
uv · run · python · ファイル名.py
```

VSCoideと同じ操作でPythonプログラムを実行可能
右上「▶ Run Python File」ボタンでも実行可能

数値の計算と型の確認

ai_programming_1_01_04. py

```
1  # 数値の計算とtype()で型の確認
2
3
4  def main():
5      # 整数
6      a = 10
7      print(a, type(a))
8
9      # 小数（浮動小数点数）
10     b = 3.14
11     print(b, type(b))
12
13     # 文字列
14     c = "Hello"
```

実行結果

```
10 <class 'int'>
3.14 <class 'float'>
Hello <class 'str'>
3.3333333333333335 <class 'float'>
3 <class 'int'>
```

- `type()`: データの型を確認する関数
- `int`: 整数、`float`: 小数、`str`: 文字列
- .pyファイル実行時のおまじない
 - `def main():`
 - `if __name__ == "__main__":`

数値の計算と型の確認

ai_programming_1_01_04. py

```
13     ... # 文字列
14     ... c = "Hello"
15     ... print(c, type(c))
16
17     ... # 計算結果の型
18     ... d = 10 / 3
19     ... print(d, type(d))
20
21     ... e = 10 // 3
22     ... print(e, type(e))
23
24
25 if __name__ == "__main__":
26     ... main()
```

実行結果

```
10 <class 'int'>
3.14 <class 'float'>
Hello <class 'str'>
3.3333333333333335 <class 'float'>
3 <class 'int'>
```

- `type()`: データの型を確認する関数
- `int`: 整数、`float`: 小数、`str`: 文字列
- .pyファイル実行時のおまじない
 - `def main():`
 - `if __name__ == "__main__":`

3つの環境の比較

	Google Colab	VSCoide	Cloud Shell Editor
動作場所	ブラウザ（クラウド）	ローカルPC	ブラウザ（クラウド）
ファイル形式	.ipynb, .py	.py, .ipynb	.py, .ipynb
インストール	不要	必要	不要
パッケージ管理	pip	uv	uv
特徴	セル実行・結果表示	本格的な開発環境	ブラウザ上のVSCoide

3つの環境の比較

	Google Colab	VSCode	Cloud Shell Editor
動作場所	ブラウザ（クラウド）	ローカルPC	ブラウザ（クラウド）
ファイル形式	.ipynb, .py	.py, .ipynb	.py, .ipynb
インストール	不要	必要	不要
パッケージ管理			

Colabの注意事項

- セッション切れでランタイム上のファイルが消失
- Google Driveマウントで永続化可能
- 無料版はGPU/TPU利用に制限あり

Cloud Shell Editorの注意事項

- \$HOME配下は永続（120日間非アクティブでリセット）
- ストレージ上限: 5GB
- 週あたりの使用時間に制限あり

まとめ

Google Colab
メイン環境
ノートブック形
式

VSCode
ローカル環境
.pyファイル開発

**Cloud Shell
Editor**
ブラウザ上の
VSCode
.pyファイル開発

3種類の環境を使い分けて
同じPythonプログラムを実行できることを確認